Soft assignment — formula brief (ρ)

Front of sheet / slide notes. Put the equation on the PPT; use the arrows/labels below for each symbol. **Back of sheet:** how π_{ij} is used in seating (next heading — new PDF page).

The equation (on the slide)

$$\pi_{ij} \;\propto\; \exp\left(-\rho \cdot \frac{(A_i - T_j)^2}{2\sigma^2}\right)$$

In words: "How much does player *i* prefer team *j*?" Larger weight → more likely to sit on that team (among teams that still have open seats).
Score is separate: After everyone is seated, Pass A uses $S_i = A_i - w \cdot L_C$. This equation is **only** the ASSIGN step (Pass B knob).

Arrow map — every symbol

Symbol	Say it	What it is	Briefing line
π_{ij}	"pi-i-j" / assignment weight	Soft preference of **player *i*** for **team *j*** before we normalize to probabilities and fill seats	"Not a probability yet — a preference score for that seat."
$\propto$	"proportional to"	Weights are scaled to sum to 1 over *open* teams later	"Relative preference; seats that are full get zeroed out."
$\exp(\dots)$	"e to the …"	Soft falloff: small distance → high weight; large distance → near zero	"Never a hard yes/no; always a smooth preference."
A_i	"A-sub-i"	**Ability** of synthetic player *i* (drawn talent)	"Who this person is on the talent scale."
T_j	"T-sub-j"	**Team target** for team *j* (one number per team)	"The 'ideal seat' that team is aiming to fill."
$A_i - T_j$	"ability minus target"	Mismatch between this player and that team	"How far is this person from that team's target?"
$(A_i - T_j)^2$	"mismatch squared"	Same mismatch, always positive; big mismatches hurt a lot	"Far away is punished more than a little off."
ρ	"rho"	**Assortativity knob** (Pass B)	"**This is what we turn.** Higher ρ → only near-matches get weight."
σ	"sigma"	Fixed **length-scale** in ability units (default ~0.65)	"How wide 'near' is. Held fixed while we sweep ρ."
$2\sigma^2$	"two sigma-squared"	Denominator that sets the width of the soft match	"With σ fixed, only ρ changes how sharp the match is."

What moving ρ does (one sentence each)

- **ρ = 0** → every open team looks the same (max mixing).
- **ρ small (e.g. 0.1)** → mild preference for $T_j \approx A_i$; still a lot of overlap across teams.
- **ρ moderate (e.g. 1)** → clear soft sorting; talent windows still overlap (like real college forensics).
- **ρ large (e.g. 8, 32)** → sharp assortativity; almost only near-matches sit together.
- **Not on this formula: sort-and-chop** is a *different* hard rule (sort by ability, cut into slices). It is **not** "ρ → ∞."

Suggested PPT layout

1. **Big equation** centered.
2. **Arrows** from ρ, A_i, T_j, σ, π_{ij} to one-line callouts (use the "Briefing line" column).
3. **Footer:** "Pass B varies ρ only; score S_i and top-K stay fixed."

Where this lives in code

- Engine: `sports/tier1_pool_assignment.py` → `_kernel_weights` / `soft_assign`
- Experiment: `sports/scripts/540_rho_ablation_bundle.py` (Pass B)
- Plain English: `3-Master_Plan/re_entry/04_Pass_A_and_Pass_B_in_Plain_English.md`

Soft assignment — how π_{ij} seats players

Back of sheet. The equation on the front is only the preference kernel. This page is the algorithm that *uses* those weights.

We already have every player's ability A_i and every team's target T_j. Soft assignment does **not** solve a big matching problem all at once. It walks through players **one at a time** and seats each person on **exactly one** team that still has an open roster slot.

Order. Players are shuffled first. That matters: if we always seated high-A people first, early choosers would lock elite teams and later players would be leftovers. Random order keeps the fill process from being ability-sorted by accident.

For each player i:

Compute a preference weight toward **every** team j from the soft kernel — that is your π_{ij} idea (still unnormalized). Teams whose T_j is close to A_i get high weight; far teams get low weight. Raising ρ makes that contrast sharper; $\rho = 0$ makes every open team look the same.

Capacity. Any team already at full roster size gets its weight set to **zero**. You cannot sit there no matter how good the match is.

Optional extras (usually off in 540). Preferential attachment can multiply weights by how full a team already is; default $\alpha = 0$ turns that off.

Draw a seat. The remaining weights are turned into probabilities that sum to 1. Then we **sample** one team from that distribution and put player i there. So π_{ij} is not stored as a final matrix; it is used **momentarily** as the chance of landing on team j given who is still open.

Repeat until everyone is seated. Every team ends with the same roster size. The function returns `pool_id[i]` — the team index for each player — and that roster table is what later steps use to build LOO congestion and scores.

Pass B's role: same A_i / T_j draw, same score and top-K later; only this seating rule's ρ (or the hard sort-and-chop arm) changes who sits with whom.

Code: `sports/tier1_pool_assignment.py` → `_kernel_weights` (build weights) → `soft_assign` (mask full teams, normalize, sample).